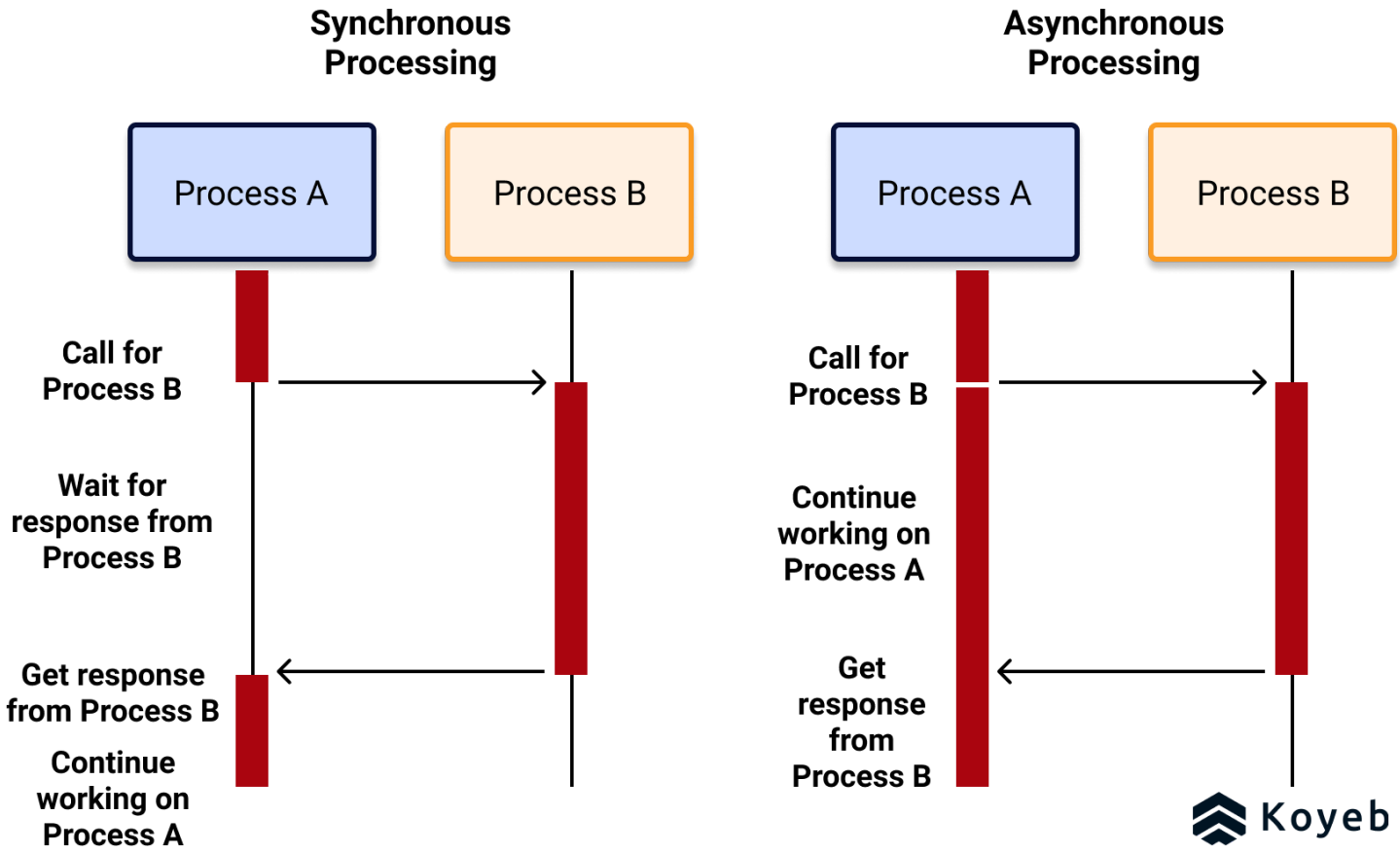
Topic: Synchronous and Asynchronous Programming in Node.js

Learning Objectives



By the end of this session, students will be able to:

- Compare asynchronous and synchronous programming models.
- Explain the asynchronous programming model using **callbacks**, **Promises**, and **async/await** in Node.js.
- Describe the synchronous programming model in Node.js.
- Explain the **event loop architecture** in Node.js.

1. Introduction to Programming Models

Synchronous Programming

Definition:

Synchronous programming executes tasks **one at a time, in sequence**. Each task must complete before the next one starts.

Analogy:
Imagine standing in line at a bank — you must wait for the person in front of you to finish before it's your turn.

Example:

```
console.log("Task 1");
console.log("Task 2");
console.log("Task 3");
```

Output:

```
Task 1
Task 2
Task 3
```

➡️ Each line executes only after the previous one has completed.

Real-life scenario:
Reading a large file synchronously blocks the main thread — your program cannot process any other requests until the file operation finishes.

Example:

```
const fs = require("fs");
const data = fs.readFileSync("data.txt", "utf8");
console.log(data);
console.log("File read complete.");
```

If `data.txt` is large, everything else waits until it's done reading.

Asynchronous Programming

Definition:
Asynchronous programming allows multiple operations to happen **without waiting for each other to finish**. Tasks can start, continue in the background, and notify the program when they're done.

Analogy:
It's like ordering food at a restaurant — you place your order and can do other things (like chatting or checking your phone) while waiting for your food.

Why Asynchronous?

Node.js is single-threaded but uses **non-blocking I/O**, which allows it to handle many operations efficiently — especially useful for servers handling multiple requests.

2. Asynchronous Programming Models in Node.js

A. Callbacks

Definition:

A callback is a function passed as an argument to another function to be executed later when a task is done.

Syntax Example:

```
function fetchData(callback) {
  setTimeout(() => {
    console.log("Data fetched!");
    callback();
  }, 2000);
}

fetchData(() => {
  console.log("Processing data...");
});
```

Output:

```
Data fetched!
Processing data...
```

Problem:

If multiple asynchronous operations depend on each other, callbacks can nest deeply — this is called **callback hell**.

Example of Callback Hell:

```
getUser(id, (user) => {
  getPosts(user, (posts) => {
    getComments(posts, (comments) => {
      console.log("Done!");
    });
  });
});
```

B. Promises

Definition:

A **Promise** is an object that represents a value which may be available **now, later, or never**.
It simplifies handling asynchronous operations.

States of a Promise:

- *Pending*
- *Fulfilled*
- *Rejected*

Syntax Example:

```
const fetchData = new Promise((resolve, reject) => {
  setTimeout(() => {
    const success = true;
    success ? resolve("Data fetched!") : reject("Error fetching data");
  }, 2000);
});

fetchData
  .then((result) => console.log(result))
  .catch((error) => console.error(error));
```

Output:

```
Data fetched!
```

Applied Example: Reading a File Asynchronously

```
const fs = require("fs").promises;

fs.readFile("data.txt", "utf8")
  .then((data) => console.log(data))
  .catch((err) => console.error("Error:", err));
```

C. Async/Await

Definition:

`async` and `await` make asynchronous code look and behave more like synchronous code.

- `async` marks a function as asynchronous.
- `await` pauses the execution until the Promise settles.

Example:

```
function fetchData() {
  return new Promise((resolve) => {
    setTimeout(() => resolve("Data received"), 2000);
  });
}

async function processData() {
  console.log("Fetching data...");
  const data = await fetchData();
  console.log(data);
  console.log("Processing complete!");
}

processData();
```

Output:

```
Fetching data...
Data received
Processing complete!
```

Applied Example: File Reading with Async/Await

```
const fs = require("fs").promises;

async function readFileAsync() {
  try {
    const data = await fs.readFile("data.txt", "utf8");
    console.log(data);
  } catch (err) {
    console.error("Error reading file:", err);
  }
}

readFileAsync();
```

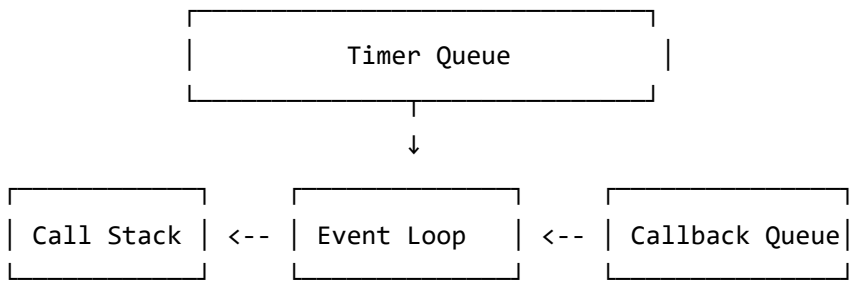
3. The Event Loop in Node.js

Definition:
The **event loop** is the mechanism that allows Node.js to perform non-blocking I/O operations — it handles multiple events asynchronously on a single thread.

Working Process:

1. Node.js starts with a call stack and a message queue.
2. When asynchronous tasks complete, their callbacks go into the queue.
3. The event loop pushes queued callbacks into the call stack when it's empty.

Visualization:



Example:

```
console.log("Start");

setTimeout(() => {
  console.log("Inside Timeout");
}, 0);

console.log("End");
```

Output:

```
Start
End
Inside Timeout
```

Even though `setTimeout` has 0ms delay, it goes to the **event queue** and executes **after** the main code completes.

4. Comparison Table

Feature	Synchronous	Asynchronous
Execution	One task at a time	Multiple tasks concurrently
Blocking	Yes	No
Performance	Slower for I/O operations	Faster and efficient
Code Example	`fs.readFileSync()`	`fs.readFile()` or `fs.promises.readFile()`

5. Class Activity / Classwork

Classwork 1: Callback Practice

Create a function that reads a file asynchronously using a callback and logs “Done reading file!” after the data is displayed.

Classwork 2: Promises

Convert your callback-based file reader into a version that uses Promises.

Classwork 3: Async/Await

Write a function using `async/await` that fetches JSON data from an API (e.g., https://jsonplaceholder.typicode.com/users) and logs it neatly.

Classwork 4: Event Loop Understanding

Predict and explain the output:

```
console.log("A");
setTimeout(() => console.log("B"), 0);
Promise.resolve().then(() => console.log("C"));
console.log("D");
```

Expected Output:

```
A
D
C
B
```

✅ Discuss *why* — because **Promises** are handled in the **microtask queue**, which has higher priority than the **callback queue**.